Lab 6: Putting It All Together

C++ Programming: Chapters 1–4 Review

Topics

- Variables and Data Types
- Input and Output
- Arithmetic Operations
- Type Conversion
- Conditional Statements
- switch Statements
- Functions
- Formatted Output

Objective:
Build a complete C++ program that combines the major concepts covered so far.

From Small Programs to Larger Programs

So far, we have written programs that perform individual tasks:

- ```
  cin >> number;
  ```

- ```
  result = num1 + num2;
  ```

- ```
  if (score >= 90)
  {
    cout << "A";
  }
  ```

- ```
  int sum(int a, int b)
  {
    return a + b;
  }
  ```

We have combined these smaller pieces of code in previous labs. Today, we will go further by combining more concepts into one complete program.

The Problem: Student Grade Calculator

1. Write a program that:
2. Asks for the student's name.
3. Accepts scores for:
    1. Exam 1
    2. Exam 2
    3. Homework
    4. Lab
4. Calculates the exam average.
5. Calculates the student's final numerical grade.
6. Determines the student's letter grade.
7. Displays a message based on the letter grade.
8. Displays a formatted grade report.

The formula is:

```
Final Grade = (Exam Average × 0.50) + (Homework × 0.25)
                                    + (Lab × 0.25)
```

Program Structure

```
Get student information
        ↓
Get scores
        ↓
Calculate exam average
        ↓
Calculate final grade
        ↓
Determine letter grade
        ↓
Display performance message
        ↓
Display final report
```

Instead of putting everything inside `main()`, we can use functions.

Why Use Functions?

- Functions allow us to divide a large problem into smaller problems. Instead of:

```
int main()
{
    // hundreds of lines doing everything
}
```

Functions just breaks a larger code into smaller chunks of codes.

  - we can have:

```
double getScore();
double calculateExamAverage(double exam1, double exam2);
char getLetterGrade(double grade);
void displayMessage(char grade);
```

Each function has one specific job.

Function Review

- A function can:
  - Receive information through parameters
  - Perform a task
  - Return a value

Example:

```
double calculateAverage(double score1, double score2)
{
    return (score1 + score2) / 2.0;
}
```

score1, score2 → parameters
exam1, exam2 → arguments

Function call:

```
double average = calculateAverage(exam1, exam2);
```

Parameters appear in the function definition; arguments are the actual values supplied when the function is called.

Getting Input with a Function

- Instead of repeatedly writing:

```
cout << "Enter score: ";
cin >> score;
```

- we can create:

```
double getScore()
{
    double score;

    cout << "Enter score: ";
    cin >> score;

    return score;
}
```

- Then:

```
exam1 = getScore();
exam2 = getScore();
homework = getScore();
lab = getScore();
```

This saves us time and save

Calculate the Exam Average

- Create a function:

```
double calculateExamAverage(double exam1, double exam2)
{
    return (exam1 + exam2) / 2.0;
}
```

- If:
  Exam 1 = 80
  Exam 2 = 90
- Then
  Exam Average = 85.0

Question: Why do we use: `2.0` instead of: `2`

With double operands either works here, but 2.0 makes it clear that we are performing floating-point arithme

Weighted Final Grade

- For this lab, use:

Category	Weight
Exams	50%
Homework	25%
Lab	25%

The formula is:

```
Final Grade = (Exam Average × 0.50) + (Homework × 0.25) + (Lab × 0.25)
```

- Function:

```
double calculateFinalGrade(double examAverage,
                           double homework,
                           double lab)
{
    return (examAverage * 0.50)
         + (homework * 0.25)
         + (lab * 0.25);
}
```

Determine the Letter Grade

- Use if / else if / else:

```
char getLetterGrade(double grade)
{
    if (grade >= 90)
        return 'A';
    else if (grade >= 80)
        return 'B';
    else if (grade >= 70)
        return 'C';
    else if (grade >= 60)
        return 'D';
    else
        return 'F';
}
```

- Why is if/else-if appropriate here?

Because we are checking ranges of values.

Using switch

- Once we have a letter grade:
  ```
  char letterGrade = getLetterGrade(finalGrade);
  ```
- we can use switch:

```
switch (letterGrade)
{
    case 'A':
        cout << "Excellent performance!" << endl;
        break;

    case 'B':
        cout << "Good performance!" << endl;
        break;

    case 'C':
        cout << "Satisfactory performance." << endl;
        break;

    case 'D':
        cout << "Needs improvement." << endl;
        break;

    case 'F':
        cout << "Failing." << endl;
        break;
}
```

Why switch here?

We are comparing specific values:
`'A', 'B', 'C', 'D', 'F'`

`if` vs. `switch`

if / else if	**switch**
Good for ranges	Good for specific values
grade >= 90	case 'A':
Can use <, >, <=, >=	Matches individual values
Complex conditions	Menus and selections

- In our program: `if (grade >= 90)` determines the letter grade.
- Then: `switch (letterGrade)` determines the message.

Formatting the Output

- Include:

`#include <iomanip>` for `cout << fixed << setprecision(2);`

- This displays: `85.00 92.50 87.25` instead of inconsistent numbers of decimal places.
- Example:

```
cout << "Final Grade: "
     << fixed << setprecision(2)
     << finalGrade << endl;
```

- Student Name:

  Because a student's name may contain spaces:

```
string name;

cout << "Enter student name: ";
getline(cin, name);
```

For example: `Ebuka Oguchi` Using: `cin >> name;` would only read: `Ebuka` when whitespace separates the in

Functions We Need

- Our program will contain:

```
double getScore();

double calculateExamAverage(double exam1,
                            double exam2);

double calculateFinalGrade(double examAverage,
                           double homework,
                           double lab);

char getLetterGrade(double grade);

void displayMessage(char grade);
```

- Notice that some functions return values: `double` `char`
- while: `void` means the function does not return a value.

Function Prototypes

- If our function definitions are placed after `main()`, we can place prototypes before `main()`:

```
double getScore();

double calculateExamAverage(double, double);

double calculateFinalGrade(double, double, double);

char getLetterGrade(double);

void displayMessage(char);
```

  - Then:

```
int main()
{
    // function calls
}
```

- And the function definitions can appear afterward.

Building `main()`

- Now main() can coordinate the entire program:

```
int main()
{
    string name;
    double exam1, exam2;
    double homework, lab;

    cout << "Enter student name: ";
    getline(cin, name);

    cout << "Exam 1" << endl;
    exam1 = getScore();

    cout << "Exam 2" << endl;
    exam2 = getScore();

    cout << "Homework" << endl;
    homework = getScore();

    cout << "Lab" << endl;
    lab = getScore();

    // More code goes here

    return 0;
}
```

Perform the Calculations

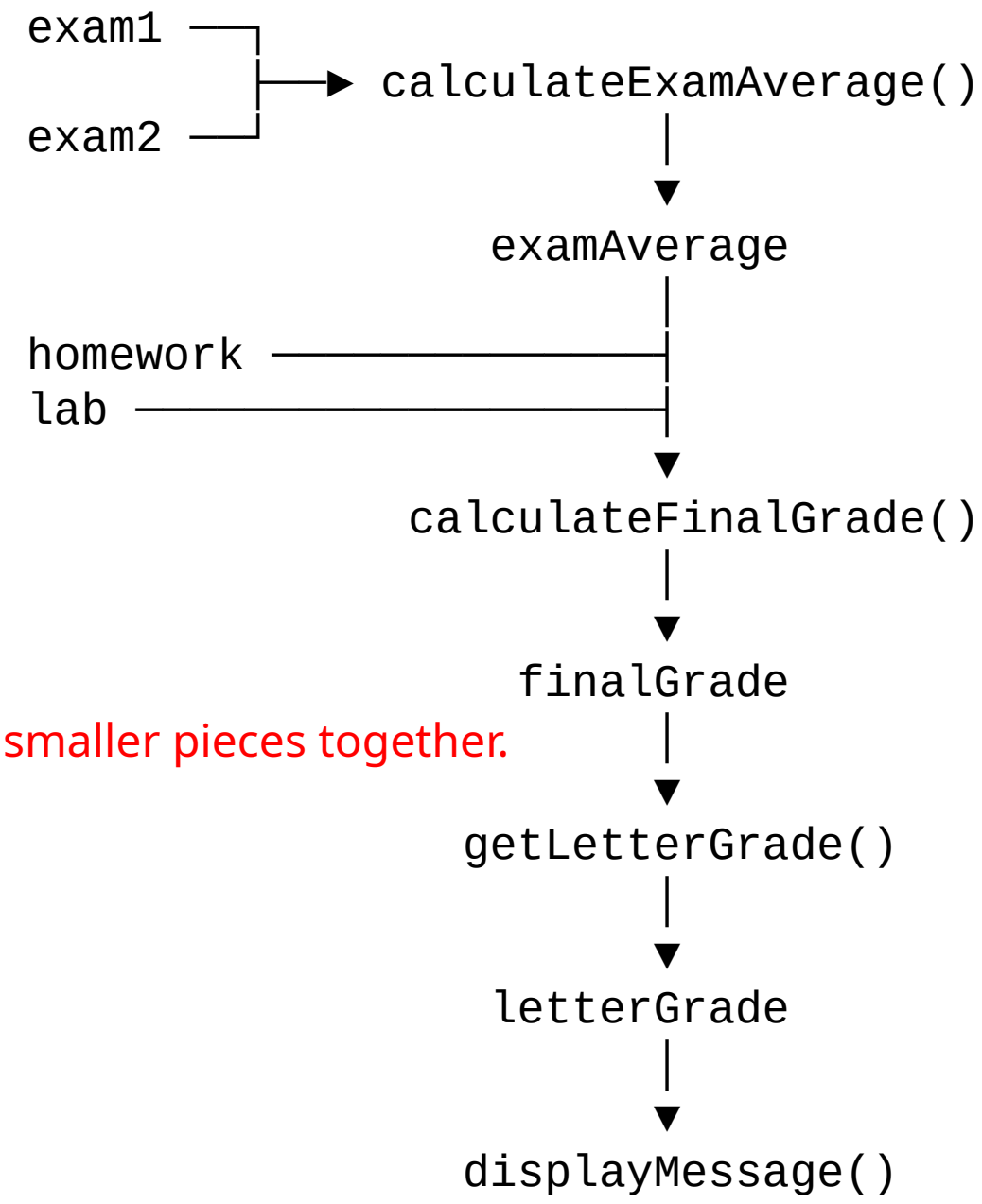
Inside `main()`:

```
double examAverage =
    calculateExamAverage(exam1, exam2);

double finalGrade =
    calculateFinalGrade(examAverage,
                        homework,
                        lab);

char letterGrade =
    getLetterGrade(finalGrade);
```

Program Flow



A large program is built by connecting smaller pieces together.

Student Grade Calculator Code

```
#include <iostream>
#include <iomanip>
#include <string>


using namespace std;


// Function prototypes
double getScore();
double calculateExamAverage(double, double);
double calculateFinalGrade(double, double, double);
char getLetterGrade(double);
void displayMessage(char);



int main(){
string name;
double exam1, exam2;
double homework, lab;
double examAverage;
double finalGrade;
char letterGrade;


// Get student's name
cout << "Enter student name: ";
getline(cin, name);
// Get scores
cout << "\nExam 1" << endl;
exam1 = getScore();


cout << "\nExam 2" << endl;
exam2 = getScore();


cout << "\nHomework" << endl;
homework = getScore();

cout << "\nLab" << endl;
lab = getScore();
// Calculate exam average
examAverage = calculateExamAverage(exam1, exam2);
// Calculate final grade
finalGrade = calculateFinalGrade(examAverage, homework, lab),


// Determine letter grade
letterGrade = getLetterGrade(finalGrade);


// Format decimal output
cout << fixed << setprecision(2);
```

```
// Display grade report
cout << "\n-------- GRADE REPORT -----" << endl;
cout << "Student: " << name << endl;
cout << "Exam Average: " << examAverage << endl;
cout << "Homework: " << homework << endl;
cout << "Lab: " << lab << endl;
cout << "Final Grade: " << finalGrade << endl;
cout << "Letter Grade: " << letterGrade << endl;

displayMessage(letterGrade);

return 0;
}



// Get a score from the user
double getScore()
{
double score;



cout << "Enter score: ";
cin >> score;

if (score < 0 || score > 100)
{
cout << "Invalid score." << endl;
}
return score;
}
```

```
// Calculate exam average
double calculateExamAverage(double exam1, double exam2)
{
return (exam1 + exam2) / 2.0;
}



// Calculate weighted final grade
double calculateFinalGrade(double examAverage,
                           double homework, double lab){
return (examAverage * 0.50) + (homework * 0.25) + (lab * 0.25);
}

//Determin letter grade
char getLetterGrade(double grade){
if (grade >= 90){
return 'A';
}
else if (grade >= 80){
return 'B';
}
else if (grade >= 70){
return 'C';
}
else if (grade >= 60){
return 'D';
}
else {
return 'F';
}
}
```

```
// Display performance message
void displayMessage (char grade)
{
switch (grade)
{
case 'A':
cout << "Excellent performance!" << endl;
break;

case 'B':
cout << "Good performance!" << endl;
break;

case 'C':
cout << "Satisfactory performance." << endl;
break;

case 'D':
cout << "Needs improvement." << endl;
break;

case 'F':
cout << "Failing." << endl;
break;

default:
cout << "Invalid grade." << endl;
}
}
```

Expected Output

```
Enter student name: Jane Smith

Exam 1
Enter score: 85

Exam 2
Enter score: 92

Homework
Enter score: 88

Lab
Enter score: 95

------ GRADE REPORT ------

Student: Jane Smith
Exam Average: 88.50
Homework: 88.00
Lab: 95.00
Final Grade: 90.00
Letter Grade: A

Excellent performance!
```

Using the stated weights, the final grade for those values is exactly `90.00`

In-Class Exercise: Shopping Discount Calculator

- Write a C++ program that simulates a shopping discount calculator.
- Your program should ask the user to enter:
  - Customer name
  - Price of an item
  - Quantity purchased
  - Customer type

- The customer types and discounts are:

Customer Type	Code	Discount
Regular Customer	R	0%
Student	S	10%
Member	M	15%

Program Requirements

ur program must:

- Ask the user for the customer's name.
- Ask for the price of the item.
- Ask for the quantity purchased.
- Ask for the customer type `(R, S, or M)`.
- Calculate the subtotal: `Subtotal = Price × Quantity`
- Determine the appropriate discount rate based on the customer type.
- Calculate the discount amount: `Discount Amount = Subtotal × Discount Rate`
- Calculate the final total: `Final Total = Subtotal − Discount Amount`
- Use an `if / else if / else` statement to determine the discount rate.
- Use a switch statement to display an appropriate message based on the customer type.
- Use user-defined functions to organize your program. Your program must contain at least three functions in addition to n
- Use `fixed` and `setprecision(2)` to display monetary values to two decimal places.

Example

```
Enter customer name: Jane Smith
Enter item price: $50
Enter quantity: 3
Enter customer type (R, S, or M): S

------ PURCHASE SUMMARY ------
Customer: Jane Smith
Subtotal: $150.00
Discount: $15.00
Final Total: $135.00
Student Discount Applied
```

Required C++ Concepts

- ✓ Variables and data types
- ✓ string, int, double, and char
- ✓ cin / cout
- ✓ getline()
- ✓ Arithmetic operators
- ✓ Functions
- ✓ Parameters and arguments
- ✓ Return values
- ✓ if / else if / else
- ✓ switch / case / break
- ✓ fixed and setprecision()

Do not place the entire program inside `main()`.

Challenge: Input Validation

- What happens if the user enters: `Item price: -50` or `Quantity: -3`
- Detects an invalid input:

```
if (price <= 0 || quantity <= 0)
{
    cout << "Error: Price and quantity must be greater than zero." << endl;
}
```

Think Before You Code

- Before writing the complete program, identify:
- Inputs
  - `Customer name`
  - `Item price`
  - `Quantity`
  - `Customer type`
- Processing
  - `Calculate subtotal`
  - `Determine discount rate`
  - `Calculate discount amount`
  - `Calculate final total`
- Outputs
  - `Customer name`
  - `Subtotal`
  - `Discount amount`
  - `Final total`
  - `Customer-type message`

Then determine: Which parts should become functions?

Key Takeaway

A program does not have to be one large block of code.

```
Large Problem
      ↓
Break into smaller problems
      ↓
Create functions
      ↓
Connect the functions in main()
      ↓
Complete Program
```

- Good programs are organized into smaller, understandable pieces.

Additional Practice Questions

Practice 1: Car Rental Calculator

Write a C++ program that calculates the cost of renting a car.

Ask the user to enter:

- Customer name
- Number of rental days
- Daily rental rate
- Vehicle type (E, S, or L)

Vehicle types:

Vehicle	Code	Additional Fee
Economy	E	0%
SUV	S	10%
Luxury	L	20%

Example:

```
Enter customer name: Jane Smith
Enter rental days: 4
Enter daily rate: $50
Enter vehicle type (E, S, or L): S

------ RENTAL SUMMARY ------
Customer: Jane Smith
Base Cost: $200.00
Additional Fee: $20.00
Final Cost: $220.00
SUV Selected
```

Program Requirements

1. Ask for the customer's name.
2. Ask for the number of rental days.
3. Ask for the daily rental rate.
4. Ask for the vehicle type.
5. Calculate:
   `Base Cost = Number of Days × Daily Rate`
6. Use if / else if / else to determine the additional-fee rate.
7. Calculate:
   `Additional Fee = Base Cost × Fee Rate`
8. Calculate:
   Final Cost = Base Cost + Additional Fee
9. Use a switch statement to display the selected vehicle type.
10. Use at least three user-defined functions in addition to mair
11. Use `fixed` and `setprecision(2)` for monetary values.

Practice 2: Shipping Cost Calculator

- Write a C++ program that calculates the cost of shipping a package.
- Ask the user to enter:
  - Customer name
  - Package weight
  - Shipping distance
  - Shipping type (S, E, or O)

Shipping types:

Shipping Type	Code	Rate per Mile
Standard	S	$0.50
Express	E	$0.75
Overnight	O	$1.00

Example:

Enter customer name: John Brown
Enter package weight: 25
Enter shipping distance: 100
Enter shipping type (S, E, or O): E
------ SHIPPING SUMMARY ------
Customer: John Brown
Shipping Cost: $75.00
Heavy-Package Fee: $10.00
Final Cost: $85.00
Express Shipping

Program Requirements

1. Ask for the customer's name.
2. Ask for the package weight.
3. Ask for the shipping distance.
4. Ask for the shipping type.
5. Use `if / else if / else` to determine the rate per mile.
6. Calculate the shipping cost: Final Cost = Shipping Cost + Heavy-Package Fee
7. Use a switch statement to display the selected shipping type.
8. Use at least three user-defined functions in addition to `main()`.
9. Use `fixed` and `setprecision(2)` for monetary values.